

Tư tưởng tối ưu hóa một lớp bài toán tìm kiếm: sắp thứ tự dữ liệu

Liu Yiming

2003

Nguồn gốc: Optimization ideas for a class of search problems: ordering data

IOI China candidate team papers / 2003 / Liu Yiming / Liu Yiming.doc

1 Dẫn nhập

Nội dung trình bày là một tư tưởng tối ưu hóa cho một lớp bài toán tìm kiếm: **sắp thứ tự dữ liệu**. Ý tưởng của phương pháp này là biến dữ liệu vốn lộn xộn thành dữ liệu có trật tự thông qua phân loại và sắp xếp đơn giản, từ đó tăng tốc quá trình tìm kiếm.

Vì sao cần làm như vậy? Trong các bài toán tìm kiếm thường gặp, dữ liệu hay có một đặc điểm lớn: lộn xộn và không có thứ tự. Tuy nhiên, trong cùng một bài, khi thuật toán sử dụng là như nhau, mức độ có thứ tự của dữ liệu khác nhau thường dẫn đến chênh lệch đáng kể về hiệu suất chương trình.

Ví dụ đầu tiên là bài toán xếp hàng vào container. Điểm khác với bài toán xếp hàng thông thường là đề yêu cầu **lắp đầy** container, nghĩa là tổng thể tích hàng hóa phải đúng bằng thể tích container. Khi so sánh hai thuật toán, ta có thể thấy thuật toán thứ hai chạy tốt trong đa số trường hợp, còn thuật toán thứ nhất không lý tưởng.

Nguyên nhân chính nằm ở cắt tỉa tối ưu. Khi dùng cắt tỉa theo tính tối ưu, ta hy vọng sớm thu được một nghiệm tốt gần nghiệm tối ưu. Nếu không sắp xếp mà tìm kiếm trực tiếp, nghiệm ban đầu rất có thể không lý tưởng. Nếu sắp xếp trước rồi mới tìm kiếm, xác suất sớm gặp một nghiệm tốt thường cao hơn; đây là lợi thế của sắp thứ tự dữ liệu.

Ưu điểm của phương pháp này không chỉ có vậy. Với phần lớn dữ liệu, nó có tác dụng tối ưu tốt, dù vẫn có dữ liệu chuyên nhắm vào nó. Nó cũng rất dễ cài đặt; với ví dụ trên, trước khi tìm kiếm chỉ cần thêm vài dòng sắp xếp nổi bọt. Hơn nữa, nó không xung đột với các phương pháp tối ưu khác, thậm chí còn tạo điều kiện thuận lợi cho chúng.

2 Hai loại sắp thứ tự dữ liệu

Sắp thứ tự dữ liệu có thể chia đại khái thành hai loại:

- sắp thứ tự dữ liệu trong giai đoạn tiền xử lý;
- sắp thứ tự dữ liệu trong giai đoạn xử lý thời gian thực.

3 Sắp thứ tự trong tiền xử lý

Thông thường, khi giải một bài toán, ta đọc dữ liệu vào rồi trực tiếp xử lý. Sắp thứ tự dữ liệu trong giai đoạn tiền xử lý nghĩa là thêm một bước trước khi xử lý chính: biến dữ liệu từ trạng thái lộn xộn thành

trạng thái có trật tự.

Ví dụ được dùng là bài *Blocks* của IOI 2000. Cách làm truyền thống dựa trên thuật toán trong không gian ba chiều. Phương pháp đó dễ nghĩ, nhưng hiệu suất không cao; một số dữ liệu chính thức thậm chí có thể gây quá thời gian. Ta thử tối ưu tìm kiếm bằng cách sắp thứ tự dữ liệu ngay trong tiền xử lý.

3.1 Sắp thứ tự cấu hình

Trước hết, xử lý có thứ tự dữ liệu của cấu hình. Sắp xếp tất cả các khối lập phương nhỏ của cấu hình theo thứ tự trong không gian rồi đánh số chúng. Dùng một tập $[1, v]$ để biểu diễn cấu hình. Như vậy, hình học ba chiều ban đầu được chuyển thành một tập một chiều.

3.2 Sắp thứ tự các khối

Sau đó xử lý có thứ tự dữ liệu của các khối có thể đặt vào. Liệt kê mọi khối có thể chèn vào cấu hình. Với mỗi khối, dùng tập các số hiệu khối lập phương mà nó chứa để biểu diễn. Khi đó việc một khối có thể đặt vào cấu hình được diễn đạt bằng quan hệ bao hàm giữa hai tập.

Chẳng hạn, nếu cấu hình hiện tại là $[1, 10]$ và lấy đi một khối gồm các ô

$$\{3, 6, 7, 9\},$$

thì cấu hình còn lại là

$$\{1, 2, 4, 5, 8, 10\}.$$

Sau tiền xử lý có thứ tự, thao tác này chỉ là phép trừ tập hợp. Nếu muốn tiếp tục đặt một khối

$$\{4, 5, 7, 8\},$$

ta kiểm tra nó có là tập con của cấu hình còn lại hay không. Vì phần tử 7 không còn trong cấu hình, khối này không thể đặt vào.

Xung đột giữa hai khối cũng được biểu diễn đơn giản: hai khối đặt riêng lẻ đều có thể hợp lệ, nhưng nếu giao của hai tập biểu diễn chúng khác rỗng, chúng không thể đặt đồng thời.

Đến đây, tiền xử lý có thứ tự đã hoàn tất. Bài toán hình học ba chiều được biến thành bài toán tìm kiếm trên tập một chiều. Phần còn lại chỉ là DFS đơn giản. Thành công của bài này chủ yếu nằm ở tiền xử lý: sắp thứ tự dữ liệu làm mô hình toán học trở nên gọn hơn.

4 Sắp thứ tự trong xử lý thời gian thực

Loại thứ hai là sắp thứ tự dữ liệu trong giai đoạn xử lý thời gian thực. Cách làm truyền thống thường là sau khi tính được dữ liệu, ta trực tiếp xử lý hoặc lưu lại. Trong cách xử lý thời gian thực có thứ tự, sau khi tính được dữ liệu, ta trước hết biến nó thành dữ liệu có trật tự, rồi mới xử lý hoặc lưu. Khi đó có thể phát hiện một số dữ liệu vô ích và loại bỏ ngay.

Một công cụ thường dùng ở đây là **biểu diễn nhỏ nhất**. Đây là một phương pháp dựa trên tư tưởng sắp thứ tự dữ liệu. Ý tưởng cơ bản là: với một lớp dữ liệu đẳng cấu, khi lưu trữ chỉ lưu đại diện nhỏ nhất.

4.1 So sánh với cách truyền thống

Với cách biểu diễn truyền thống, sau khi thu được một trạng thái hợp lệ, ta sinh ra mọi trạng thái đẳng cấu của nó. Nếu trong số đó đã có trạng thái được lưu, trạng thái hiện tại có thể bỏ; nếu không, lưu trạng thái hiện tại.

Với biểu diễn nhỏ nhất, sau khi thu được một trạng thái hợp lệ, ta chuyển nó thành biểu diễn nhỏ nhất. Nếu biểu diễn nhỏ nhất đã được lưu, bỏ trạng thái; ngược lại, lưu biểu diễn nhỏ nhất.

Biểu diễn nhỏ nhất có một tính chất quan trọng: **duy nhất**. Mỗi trạng thái có đúng một biểu diễn nhỏ nhất. Tính chất này rất có ích cho tối ưu hóa tìm kiếm.

5 Ví dụ: một biến thể của bài N quân hậu

Trước hết cần biểu diễn trạng thái. Ta dùng cách quen thuộc: biến bàn cờ hai chiều thành một bộ n phần tử, trong đó phần tử thứ i biểu diễn cột của quân hậu ở hàng thứ i .

Tiếp theo xét các phép đối xứng: lật theo trục dọc, lật theo trục ngang, lật theo đường chéo, và ba phép quay. Với $n = 5$, nếu biết bộ biểu diễn của một bàn cờ hợp lệ, ta có thể tính bộ biểu diễn sau khi lật hoặc quay bằng công thức biến đổi tọa độ tương ứng. Ở đây b_i biểu diễn cột của quân hậu ở hàng thứ i .

Dùng biểu diễn nhỏ nhất có hai cách. Cách thứ nhất là sinh trạng thái theo trình tự bình thường, chuyển nó thành biểu diễn nhỏ nhất, rồi kiểm tra. Nhưng thực ra cuối cùng ta chỉ lưu biểu diễn nhỏ nhất, nên những nghiệm không thể là biểu diễn nhỏ nhất có thể được quay lui ngay khi phát hiện.

Trong quá trình liệt kê, nếu nhận thấy trạng thái hiện tại không thể mở rộng thành biểu diễn nhỏ nhất, ta quay lui; nếu không, tiếp tục liệt kê. Điều này tạo ra một điều kiện cắt tỉa mới. Từ các công thức lật và quay, ta có thể suy ra một số điều kiện cần để tiền tố hiện tại còn có khả năng trở thành biểu diễn nhỏ nhất. Đây là điều kiện cắt tỉa khá mạnh; trong thực tế nó có thể cắt bỏ hơn một nửa số nhánh vô ích.

Nhờ tính duy nhất của biểu diễn nhỏ nhất, các nghiệm tìm được và thỏa điều kiện biểu diễn nhỏ nhất chắc chắn không đẳng cấu với nhau. Vì vậy quá trình kiểm tra đẳng cấu có thể bỏ qua; các trạng thái đã tìm thấy cũng không cần lưu lại. Không gian sử dụng giảm mạnh.

6 So sánh hai cách sắp thứ tự

Sắp thứ tự dữ liệu trong tiền xử lý thường tốn ít thời gian hơn, nhưng yêu cầu nhiều bộ nhớ hơn. Sắp thứ tự dữ liệu trong thời gian thực linh hoạt hơn, vì dữ liệu được xử lý ngay khi sinh ra, nên yêu cầu bộ nhớ nhỏ hơn. Tuy nhiên, nếu cây tìm kiếm có nhiều nút, nó không phải lúc nào cũng lý tưởng, vì có thể phải xử lý lặp lại một số nút giống nhau.

Hai phương pháp này bổ sung cho nhau. Khi áp dụng, ta có thể kết hợp cả hai để tận dụng ưu điểm và tránh nhược điểm.

7 Kết luận

Nội dung chính là biến dữ liệu hỗn loạn, vô trật tự thành dữ liệu có trật tự bằng những phương pháp đơn giản. Bản thân khoa học cũng là một vẻ đẹp; dữ liệu có trật tự phù hợp với vẻ đẹp khoa học đôi khi giúp ta đạt hiệu quả gấp bội.

Những phương pháp được trình bày ở đây không nhất thiết là phương pháp nhanh nhất cho từng bài. Điểm quan trọng là tỷ lệ hiệu quả trên chi phí: chúng chỉ cần sửa đổi một phần nhỏ chương trình ban đầu, không phải viết lại từ đầu, nên rất kinh tế. Trong thi đấu cũng như trong đời sống, điều quan trọng khi giải một vấn đề là nhanh chóng thiết kế được phương án và thu được đáp án, không phải làm cho chương trình chạy nhanh nhất bằng mọi giá. Chỉ cần giải được bài trong giới hạn thời gian, đó đã là thành công; lựa chọn hợp lý mới đem lại tỷ lệ hiệu quả cao nhất.